Adaptive Multi-Layer Embedding

Adaptive embedding is a technique that hides or inserts data by considering the texture characteristics of the host (cover) image. Instead of embedding data uniformly, the method adaptively selects suitable regions based on image texture, resulting in improved imperceptibility and higher security.

Why Is It Called "Adaptive Multi-Layer Embedding"?

The term **Adaptive Multi-Layer Embedding** is used because the framework combines:

1. **Adaptive Embedding** – Data is embedded according to local texture complexity.
2. **Multiple Security and Processing Layers** – Secret data passes through several sequential encryption, transformation, and embedding stages before being hidden inside the image.

Thus, both adaptive pixel selection and multiple protection layers are integrated into a single embedding framework.

PSNR Evaluation

Encrypted PSNR

Comparison between:

- Cover Image
- Encrypted Cover Image

Marked Encrypted PSNR

Comparison between:

- Cover Image
- Embedded (Marked) Image

Entropy Range

For an 8-bit grayscale image:

[
0 \leq H \leq 8
]

where:

- (H = 0) indicates a completely uniform image.
- (H = 8) indicates maximum randomness and uncertainty.

Overall Workflow of the Proposed Method

1. Key Generation (ECDH)

First, two secret key pairs are generated.

Using the **Elliptic Curve Diffie-Hellman (ECDH)** protocol, both communicating parties establish a shared secret key.

Purpose

- Secure key exchange
- Establishment of a confidential communication channel

2. Key Derivation (HKDF)

A single shared secret is expanded into multiple cryptographic keys using **HKDF (HMAC-based Key Derivation Function)**.

Generated keys include:

- AES key (for cover image encryption)
- ChaCha20 key (for secret image encryption)

- Random seeds for shuffling and embedding operations

Purpose

- Separation of cryptographic functions
- Multi-layer security enhancement

3. Cover Image Encryption

The cover image is protected through a two-stage process:

Step 1: Block Shuffling

The image is divided into blocks, and the blocks are randomly shuffled.

Step 2: AES-CTR Encryption

The shuffled image is encrypted using AES in Counter (CTR) mode.

Purpose

- Conceal the visual content of the cover image
- Prevent unauthorized access
- Produce a completely unreadable encrypted image

4. Secret Image Encryption

The secret image is encrypted using the **ChaCha20 stream cipher**.

Purpose

- Fast encryption
- Strong cryptographic security
- Protection of secret image content before embedding

5. Payload Creation

The encrypted secret image undergoes additional processing:

Processing Steps

1. Secret image → ChaCha20 encryption
2. Encrypted secret → Byte stream
3. Bytes → Bit conversion
4. Bit permutation (shuffling)
5. XOR operation (bit flipping)

Purpose

- Increase randomness
- Improve payload security
- Protect against statistical attacks

6. Texture-Based Pixel Selection

The cover image is analyzed to identify suitable embedding locations.

Step 1: Image Acquisition

The encrypted cover image is taken as input.

Step 2: Neighborhood Difference Calculation

Each pixel is compared with its neighboring pixels:

$$
I(i,j)-I_{neighbor}
$$

where:

- $I(i,j)$ = current pixel value

- (I_{neighbor}) = neighboring pixel value

Step 3: Texture Score Computation

Texture complexity is calculated as:

[
T(i,j)=\frac{1}{8}\sum (I(i,j)-I_{neighbor})^2
]

where:

- (T(i,j)) = texture score at pixel ((i,j))

Step 4: Smooth and Complex Region Classification

Smooth Region

Small texture values:

[
T(i,j) \rightarrow \text{Low}
]

Characteristics:

- Low variation
- Flat image areas

Complex Region

Large texture values:

[
T(i,j) \rightarrow \text{High}
]

Characteristics:

- Edges
- Detailed textures
- High local variation

Step 5: Pixel Ranking and Selection

Pixels are sorted according to their texture scores:

$$
P=\operatorname{argsort}(T)\downarrow
$$

The pixels with the highest texture scores are selected for embedding.

Advantages

- Lower visual distortion
- Better imperceptibility
- Higher embedding capacity
- Improved resistance to visual detection

7. Data Embedding

The encrypted secret bits are embedded into the selected pixels of the encrypted cover image.

Technique Used

- Adaptive BPP (Bits Per Pixel)

Purpose

- Controlled data hiding
- High embedding capacity

- Preservation of image quality

8. Data Extraction

At the receiver side, the hidden data is extracted from the stego image.

Reverse Operations

1. Extract embedded bits
2. Reconstruct payload
3. Decode shuffled and XOR-processed data

9. Secret Image Decryption

The recovered encrypted secret image is decrypted using ChaCha20.

Result

The original secret image is successfully reconstructed.

10. Cover Image Recovery

The original cover image is restored completely through the inverse operations.

Recovery Steps

1. Restore backup information
2. AES-CTR decryption
3. Block un shuffling

Result

The original cover image is recovered exactly without any loss.

Lossless Cover Recovery Achieved

11. Security and Quality Evaluation

The performance of the proposed system is assessed using several standard metrics.

PSNR (Peak Signal-to-Noise Ratio)

Measures image quality and distortion.

Higher PSNR indicates better visual quality.

SSIM (Structural Similarity Index)

Measures structural similarity between two images.

Higher SSIM indicates better preservation of image structures.

BER (Bit Error Rate)

Measures the percentage of incorrectly recovered bits.

Lower BER indicates more reliable extraction.

Entropy

Measures randomness and security strength.

Higher entropy indicates stronger resistance against statistical attacks.

Correlation Coefficient

Measures the relationship between neighboring pixels.

Lower correlation in encrypted images indicates stronger encryption performance.

Summary

The proposed Adaptive Multi-Layer Embedding framework combines:

- ECDH-based secure key exchange
- HKDF-based key derivation
- AES-CTR cover image encryption
- ChaCha20 secret image encryption
- Payload shuffling and XOR processing
- Texture-based adaptive pixel selection
- Adaptive BPP embedding
- Lossless cover image recovery
- Secure secret image reconstruction

This multi-layer design enhances **security, imperceptibility, robustness, embedding capacity, and lossless recoverability**, making it suitable for secure image steganography and reversible data hiding applications.

----------x----------

🧠 AML-APS RDHEI System — Algorithm Usage + Novelty Table

Step	Process Stage	Algorithm Used	Why This Algorithm Used	Why NOT Other Algorithms	Novelty Contribution
1	Key Generation	**ECDH (Elliptic Curve Diffie-Hellman)**	Secure shared secret generation with small key size & high security	RSA → slow, large key; DH → weaker & inefficient for modern security	🔥 Lightweight + strong cryptographic key exchange
2	Key Derivation	**HKDF-SHA256**	Produces multiple strong keys from single shared secret	Simple hash → no key separation; AES key reuse risk	🔥 Multi-key generation (AES + ChaCha + seeds)
3	Cover Encryption	**AES-CTR + Block Permutation**	Fast symmetric encryption + maintains structure for reversible processing	ECB → insecure pattern leakage; CBC → not parallel; no permutation → low diffusion	🔥 Hybrid encryption + spatial scrambling (double security layer)
4	Block Shuffling	**Random Block Permutation (PRNG-based)**	Adds spatial confusion before encryption	Fixed permutation → predictable; no permutation → weak security	🔥 Spatial security layer before cryptography
5	Secret Encryption	**ChaCha20 Stream Cipher**	High speed, secure for small payload, no padding required	AES for small data → overhead; DES → outdated; RSA → too slow	🔥 Lightweight secure secret encryption
6	Payload Construction	**Bit-level Packing + XOR + Permutation**	Ensures confidentiality + randomness of embedded data	Direct embedding → easily detectable; no shuffle → pattern leakage	🔥 Anti-pattern payload scrambling